RACKHACK

05 Intelligent Traffic Perception

Problem Explanation:

With growing urban populations and congested roads, real-time traffic monitoring is essential for road safety and efficiency. Many existing systems either detect a single object category (e.g., only vehicles) or rely on outdated sensor technology.

Your task is to develop a deep learning model that processes traffic video in real time, identifying multiple classes (vehicles, pedestrians, traffic signs, lanes, etc.), outputting structured data to help city planners, law enforcement, or autonomous vehicles.

Resources for Beginners:

- **Dataset:**
  - BDD100K: Large-scale annotated driving videos
- **Model Architectures:**
  - YOLOv8
  - Faster R-CNN
  - Detectron2
- **Deep Learning Frameworks:**
  - PyTorch
  - TensorFlow
- **Tutorials:**
  - PyTorch Object Detection

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code and a google drive link containing the weights for your Deep Learning Model (Mandatory). Deployment of your code will earn you bonus points.

1. MAP (Mean Average Precision): Accuracy for detecting multiple object classes.
2. IoU (Intersection over Union): Overlap between predicted and ground truth bounding boxes.
3. Inference Speed (FPS): Real-time performance (frames per second).
4. Robustness: Performance under diverse lighting, weather, and traffic conditions.
5. Innovation: Implementation of novel architectures (e.g., transformers, attention mechanisms).

Examples **:**

- **Traffic Sign Detection:** Detecting stop signs, speed limits, and yield signs in suburban or highway footage.
- **Vehicle Tracking:** Identifying car models, counting trucks vs. motorcycles, or tracking specific lanes.
- **Pedestrian Alerts:** Locating and tracking people in crosswalks for accident prevention.

RACKHACK

What We Expect:

- **Model Training:** PyTorch or TensorFlow
- **Data Pipeline:** Python scripts for frame extraction, data augmentation, etc.
- **Inference/Deployment:**
  - Python/Flask/FastAPI or a C++/TensorRT pipeline for real-time performance
  - Optional: ONNX or TensorRT optimization
- **Visualization:** OpenCV or a web interface to display bounding boxes on live video .

For any queries, contact - Vinamra Garg (+91 9350555053),
Luv Sharma(+91 79822 99553)